Hoisting
in Javascript

swipe

swipe

Hoisting

- Hoisting in JavaScript refers to the behavior where variable and function declarations are moved to the top of their respective scopes during the compilation phase, before the code is executed.
- This allows you to use functions and variables before they are actually declared in the code.
- However, only the declarations are hoisted, not the initializations.

swipe

Variable Hoisting

- Variables declared **using var** are hoisted to the top of their scope and initialized with undefined. This means that you can reference a var variable before its declaration, but its value will be undefined until the line of declaration and assignment is executed.

```
console.log(x); // Outputs: undefined
var x = 5;
console.log(x); // Output: 5
// The above code is interpreted as:
var x;
console.log(x);
x = 5;
console.log(x);
```

swipe

Using let and const

- Variables declared with let and const are hoisted to the top of their scope, but they are not initialized.
- They remain in a **"temporal dead zone"** (TDZ) from the start of the block until the declaration is encountered.

```
console.log(myLetVar); // ReferenceError: Cannot access 'myLetVar'
before initialization
let myLetVar = 10;
console.log(myLetVar); // Output: 10
```

swipe

Function Hoisting

- When function is declare then entire function(both the function's name and its definition) are hosted to the top of their scope.
- This allows you to call a function before its declaration.

```
greet(); // Output: Hello, world!

function greet() {
    console.log("Hello, world!");
}
// The above code is interpreted as:
function greet() {
    console.log("Hello, world!");
}
greet(); // Output: Hello, world!
```

swipe

Function Expression

- function expressions are not hoisted in the same way. The variable declared to hold the function expression is hoisted, but the function itself is not.
- If a function is called before its definition, an error occurs due to variable hoisting causing it to be undefined until the function is assigned during evaluation.

```
console.log(sayHello); // Output: undefined
sayHello(); // TypeError: sayHello is not a function

var sayHello = function() {
    console.log("Hello, world!");
};

sayHello(); // Output: Hello, world!
```